



T.C.

**NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ**

**MÜHENDİSLİK FAKÜLTESİ**

**BİLGİSAYAR MÜHENDİSLİĞİ**

Yapay Zeka Teknikleri Dersi Projesi

Ev Fiyatlarının Regresyon ile Tahmin Edilmesi

Emre KARATOP - 220610006

# 1. Giriş

## 1.1. Problem Tanımı

Bu projede kullanılan veri seti (house\_price\_regression\_dataset.csv), bir konutun piyasa değerini etkileyen çeşitli özelliklere ait gözlemler içermektedir. Bu özellikler arasında konutun metrekaresi (Square\_Footage), yatak odası sayısı (Num\_Bedrooms), banyo sayısı (Num\_Bathrooms), yapım yılı (Year\_Built) ve mahalle kalitesi (Neighborhood\_Quality) gibi fiziksel ve konumsal nitelikler bulunmaktadır. Problem, bu nitelikleri (bağımsız değişkenler) girdi olarak kullanarak, bir evin piyasa fiyatını (House\_Price - bağımlı değişken) en doğru şekilde tahmin edebilen bir model geliştirmektir. Bu, denetimli makine öğrenimi (supervised machine learning) kategorisinde yer alan bir regresyon problemidir. Ev fiyatlarının doğru bir şekilde tahmin edilmesi; alıcılar, satıcılar, emlak yatırımcıları ve finansal kurumlar için kritik bir öneme sahiptir.

## 1.2. Amaç

Bu projenin temel amacı, ev fiyatlarını tahmin etmek için Basit Lineer Regresyon, Çoklu Lineer Regresyon, Polinomsal Regresyon ve Karar Ağacı Regresyonu olmak üzere dört farklı makine öğrenimi algoritmasını uygulamak ve karşılaştırmaktır. Bu modellerin performansları  $R^2$  (Başarı Skoru), RMSE (Hata Payı) ve Eğitim/Tahmin Süresi gibi metrikler kullanılarak değerlendirilecek ve bu veri seti için en doğru ve verimli tahmin modelinin hangisi olduğu belirlenecektir.

# 2. Gelişme

## 2.1. Kullanılan Veri Seti : Home Price

Bu çalışmada, ev fiyatı tahminlemesi için house\_price\_regression\_dataset.csv adlı veri seti kullanılmıştır. Veri seti, 5000 adet konut gözleminde (satır) ve 8 adet sütundan oluşmaktadır.

Sütunlar, 7 adet bağımsız değişken (özellik) ve 1 adet bağımlı değişken (hedef) olarak ayrılmaktadır:

Özellikler (Features):

1. Square\_Footage: Evin toplam metrekaresi.
2. Num\_Bedrooms: Yatak odası sayısı.
3. Num\_Bathrooms: Banyo sayısı.
4. Year\_Built: Evin inşa edildiği yıl.
5. Lot\_Size: Arsa büyüklüğü.
6. Garage\_Size: Garaj kapasitesi (araç sayısı).
7. Neighborhood\_Quality: Mahalle kalite skoru (nümerik bir değer).

Hedef (Target):

1. House\_Price: Evin piyasa fiyatı (tahmin edilmek istenen değer).

Veri Ön İşleme: Yapılan incelemede, veri setinin temiz olduğu, eksik (null) veri içermediği ve tüm sütunların modellemeye uygun şekilde sayısal (nümerik) formatta olduğu görülmüştür. Bu nedenle ek bir veri temizleme veya dönüştürme işlemine gerek duyulmamıştır. Modelleri eğitmek ve test etmek amacıyla veri seti, %80 eğitim ve %20 test verisi olarak ikiye ayrılmıştır.

## 2.2. Kullanılan Yöntem ve Algoritmalar

Bu çalışmada, ev fiyatlarını tahmin etmek için dört farklı denetimli öğrenme (supervised learning) regresyon algoritması uygulanmış ve karşılaştırılmıştır:

**1. Basit Lineer Regresyon** (Simple Linear Regression): Bu model, bir bağımlı değişken (Ev Fiyatı) ile yalnızca bir bağımsız değişken arasındaki ilişkiyi doğrusal bir çizgi ( $y = b_0 + b_1x$ ) kullanarak modeller. Projemizde, ev fiyatını tahmin etmede en güçlü etken olduğu varsayılan Square\_Footage (metrekare) özelliği tek başına kullanılarak bir temel (baseline) model oluşturulmuştur.

**2. Çoklu Lineer Regresyon** (Multiple Linear Regression): Basit lineer regresyonun gelişmiş bir versiyonudur. Fiyatı tahmin etmek için tüm bağımsız değişkenleri (metrekare, yatak odası sayısı, yapım yılı vb.) aynı anda kullanır. Model, her bir özelliğin fiyata olan katkısını ayrı ayrı ( $y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$ ) hesaplayarak daha kapsamlı bir tahmin yapmaya çalışır.

**3. Polinomsal Regresyon** (Polynomial Regression): Bu yöntem, değişkenler arasındaki ilişkinin düz bir çizgiden (linear) daha karmaşık, yani eğrisel (non-linear) olabileceğini varsayar. Özelliklerin  $x^2$  (karesi),  $x^3$  (küpü) veya  $x_1x_2$  (etkileşimi) gibi polinomsal versiyonlarını oluşturur ve bu yeni özellikler üzerinde bir lineer regresyon uygular. Bu sayede, örneğin metrekare arttıkça fiyatın katlanarak artması gibi karmaşık ilişkileri yakalayabilir. Projemizde 2. dereceden (degree=2) bir polinom modeli kullanılmıştır.

**4. Karar Ağacı Regresyonu** (Decision Tree Regression): Lineer olmayan bir modeldir. Veri setini, özelliklerin değerlerine göre ("metrekare > 150 mi?", "yatak odası sayısı > 3 mü?" gibi) dallara ayırarak bir ağaç yapısı oluşturur. Bir evin fiyatını tahmin etmek için bu ağaçtaki kuralları takip eder ve ulaştığı "yaprak" düğümdeki evlerin ortalama fiyatını tahmin olarak döndürür. Karmaşık ilişkileri ve özellik etkileşimlerini öğrenmede etkilidir.

## 3. Sonuç ve Değerlendirme

### 3.1. Linner Regresyon Sonuçları

Bu model, ev fiyatlarındaki değişimin %98.3'ünü ( $R^2 = 0.9832$ ) sadece metrekare özelliğini kullanarak açıklamayı başarmıştır. Bu, ev fiyatlarının belirlenmesinde Square\_Footage özelliğinin çok baskın bir faktör olduğunu göstermektedir. Ancak, 32,885'lük hata payı (RMSE), tek başına yeterli olmadığını da kanıtlamaktadır.

### 3.2. Multiple Lineer Regresyon Sonuçları

Bu modelde tüm 7 özelliği dahil etmek, başarıyı %99.84'e ( $R^2 = 0.9984$ ) yükseltmiş ve hata payını 10,071'a düşürmüştür. Bu, yatak odası sayısı, banyo sayısı ve yapım yılı gibi diğer özelliklerin de fiyat üzerinde önemli bir etkisi olduğunu ve modelin başarısını mükemmele yaklaştırdığını göstermektedir. Aynı zamanda en hızlı eğitilen model olmuştur.

### 3.3. Polinomal Regresyon Sonuçları

Bu model, Çoklu Lineer Regresyon ile aynı başarı skorunu ( $R^2 = 0.9984$ ) elde etmiş ancak hata payı (RMSE) ve eğitim süresi çok az da olsa artmıştır. Bu durum, veri setindeki ilişkinin zaten çok güçlü bir şekilde doğrusal (lineer) olduğunu gösterir. Eğrisel (polinomsal) özellikler eklemek, modele anlamlı bir fayda sağlamamış, aksine modeli gereksiz yere karmaşıktırıştırılmıştır.

### 3.4. Karar Ağacı Regsyon Sonuçları

Karar Ağacı, %98.5'lik ( $R^2 = 0.9850$ ) çok yüksek bir başarı skoru elde etse de, bu skor Çoklu Lineer ve Polinomsal modellerin gerisinde kalmıştır. 31,138'lik hata payı (RMSE), Basit Lineer Regresyon'a yakın olup diğer iki modelden belirgin şekilde daha yüksektir. Bu, bu özel veri setindeki güçlü doğrusal ilişkiyi yakalamada lineer modeller kadar verimli olmadığını göstermektedir.

### 3.5. Değerlendirme

Bu çalışmada, house\_price\_regression\_dataset.csv veri setini kullanarak ev fiyatlarını tahmin etmek amacıyla dört farklı regresyon modeli test edilmiştir. Modellerin performansları,  $R^2$  (Başarı), RMSE (Hata Payı) ve Eğitim/Tahmin Süresi (Hız) metriklerine göre karşılaştırılmıştır.

#### Performans Karşılaştırması

- **Başarı ( $R^2$ ):** En yüksek başarı skoruna ( $R^2 = 0.9984$ ) Çoklu Lineer Regresyon ve Polinomsal Regresyon modelleri ulaşmıştır. Bu skor, her iki modelin de ev fiyatlarındaki değişimin %99.84'ünü (neredeyse tamamını) açıklamayı başardığını göstermektedir.
- **Hata (RMSE):** En düşük hata payına, yani gerçeğe en yakın tahminlere (RMSE: 10,071.48) Çoklu Lineer Regresyon modeli sahiptir. Polinomsal Regresyon (RMSE: 10,187.32) çok yakın bir sonuç olsa da, Çoklu Lineer Regresyon'un gerisinde kalmıştır.
- Basit Lineer Regresyon ( $R^2 = 0.9832$ ) ve Karar Ağacı ( $R^2 = 0.9850$ ) modelleri de son derece yüksek başarı göstermiştir. Bu durum, kullanılan veri setinin yapısının tahmin edilmesinin çok kolay olduğunu ve özellikle Square\_Footage gibi özelliklerle fiyat arasında çok güçlü bir doğrusal ilişki bulunduğunu kanıtlamaktadır.

#### Verimlilik (Hız) Karşılaştırması

- Performansın yanı sıra hız da önemli bir faktördür. Çoklu Lineer Regresyon (0.0010 saniye), test edilen modeller arasında en hızlı eğitilen model olmuştur.
- Polinomsal Regresyon (0.0030 sn) ve Karar Ağacı (0.0034 sn) da çok hızlı olsa da, Çoklu Lineer Regresyon'dan yaklaşık 3 kat daha yavaş kalmıştır.

Tüm metrikler (başarı, hata payı ve hız) bir arada değerlendirildiğinde, bu ev fiyatı tahmini problemi için Çoklu Lineer Regresyon modeli en iyi ve en verimli model olarak belirlenmiştir.

Bu model, en yüksek  $R^2$  skorunu, en düşük RMSE hata payını ve en hızlı eğitim süresini bir arada sunmaktadır. Polinomsal Regresyon modelinin ek karmaşıklığı (yeni özellikler türetmesi), bu veri seti üzerinde anlamlı bir performans artışı sağlamamış, bu nedenle daha basit ve daha hızlı olan Çoklu Lineer Regresyon modeli tercih edilmiştir.

#### 4. Kaynak

- <https://ecemagacdograyan.medium.com/regresyon-algoritmlar%C4%B1-nedir-2-e66b12907b57>
- <https://www.kaggle.com/datasets/prokshitha/home-value-insights>
- <https://pypi.org/project/matplotlib/>

```

linear_regression.py
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import numpy as np
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 import time
9
10 # --- VERİ YÜKLEME VE AYIRMA (Her dosyada gerekli) ---
11 file_path = 'house_price_regression_dataset.csv'
12 df = pd.read_csv(file_path)
13 y = df['House_Price']
14 X = df.drop('House_Price', axis=1)
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
16 # --- BİTİ ---
17
18
19 print("--- Basit Lineer Regresyon ---")
20
21 # HATA BURADAYDI: 'Square Footage' -> 'Square_Footage' olarak düzeltildi
22 X_train_simple = X_train[['Square_Footage']]
23 X_test_simple = X_test[['Square_Footage']]
24
25 start_train = time.time()
26 simple_model = LinearRegression()
27 simple_model.fit(X_train_simple, y_train)
28 train_time = time.time() - start_train
29
30 start_pred = time.time()
31 y_pred_simple = simple_model.predict(X_test_simple)
32 pred_time = time.time() - start_pred
33
34 simple_model = LinearRegression()
35 simple_model.fit(X_train_simple, y_train)
36
37 y_pred_simple = simple_model.predict(X_test_simple)
38
39 r2_simple = r2_score(y_test, y_pred_simple)
40 mae_simple = mean_absolute_error(y_test, y_pred_simple)
41 rmse_simple = np.sqrt(mean_squared_error(y_test, y_pred_simple))
42
43 print(f"Modelin Katsayısı (b1 - Eğim): {simple_model.coef_[0]:.2f}")
44 print(f"Modelin Kesişim Noktası (b0): {simple_model.intercept_[0]:.2f}")
45 print(f"\n--- Test Seti Performans Metrikleri ---")
46 print(f"R-Kare (R2): {r2_simple:.4f}")
47 print(f"Ortalama Mutlak Hata (MAE): {mae_simple:.2f}")
48 print(f"Kök Ortalama Kare Hata (RMSE): {rmse_simple:.2f}")
49
50 print(f"\n--- Süre Metrikleri ---")
51 print(f"Eğitim Süresi: {train_time:.6f} saniye")
52 print(f"Tahmin Süresi: {pred_time:.6f} saniye")
53
54 # Grafiği oluşturma ve kaydetme
55 plt.figure(figsize=(10, 6))
56 sns.scatterplot(x=X_test_simple['Square_Footage'], y=y_test, alpha=0.6, label='Gerçek Değerler')
57 plt.plot(X_test_simple['Square_Footage'], y_pred_simple, color='red', linewidth=2, label='Regresyon Doğrusu (Tahmin)')
58 plt.title('Basit Lineer Regresyon: Metrekare vs. Ev Fiyatı (Test Seti)')
59 plt.xlabel('Metrekare (Square_Footage)')
60 plt.ylabel('Ev Fiyatı (House_Price)')
61 plt.legend()
62 plt.grid(True)
63 plt.savefig('simple_linear_regression.png')
64 print("Grafik 'simple_linear_regression.png' olarak kaydedildi.")
65 # plt.show() # Eğer grafiği ekranda görmek isterseniz bu satırı açabilirsiniz

```

```

python .\linear_regression.py
--- Basit Lineer Regresyon ---
--- Basit Lineer Regresyon ---
Modelin Katsayısı (b1 - Eğim): 200.56
Modelin Kesişim Noktası (b0): 54229.06

--- Test Seti Performans Metrikleri ---
--- Test Seti Performans Metrikleri ---
R-Kare (R2): 0.9832
R-Kare (R2): 0.9832
Ortalama Mutlak Hata (MAE): 27179.53
Kök Ortalama Kare Hata (RMSE): 32885.95

--- Süre Metrikleri ---
Eğitim Süresi: 0.015619 saniye
Tahmin Süresi: 0.000000 saniye

```

```

multiple_linear_regression.py
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import numpy as np
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 import time # Süre ölçümü için eklendi
9
10 # --- VERİ YÜKLEME VE AYIRMA ---
11 file_path = 'house_price_regression_dataset.csv'
12 df = pd.read_csv(file_path)
13 y = df['House_Price']
14 X = df.drop('House_Price', axis=1)
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
16 # --- BİTTİ ---
17
18 print("--- Çoklu Linear Regresyon (Tüm Özellikler) ---")
19
20 # --- Eğitim Süresi Ölçümü ---
21 # time.time() YERİNE time.perf_counter() KULLANILDI
22 start_train = time.perf_counter()
23 multi_model = LinearRegression()
24 multi_model.fit(X_train, y_train)
25 train_time = time.perf_counter() - start_train
26 # --- BİTTİ ---
27
28 # --- Tahmin Süresi Ölçümü ---
29 # time.time() YERİNE time.perf_counter() KULLANILDI
30 start_pred = time.perf_counter()
31 y_pred_multi = multi_model.predict(X_test)
32 pred_time = time.perf_counter() - start_pred
33 # --- BİTTİ ---
34
35 r2_multi = r2_score(y_test, y_pred_multi)
36 mae_multi = mean_absolute_error(y_test, y_pred_multi)
37 rmse_multi = np.sqrt(mean_squared_error(y_test, y_pred_multi))
38
39 print(f"\n--- Performans Metrikleri ---")
40 print(f"R-Kare (R2): {r2_multi:.4f}")
41 print(f"Ortalama Mutlak Hata (MAE): {mae_multi:.2f}")
42 print(f"Kök Ortalama Kare Hata (RMSE): {rmse_multi:.2f}")
43
44 print(f"\n--- Süre Metrikleri ---")
45 print(f"Eğitim Süresi: {train_time:.6f} saniye")
46 print(f"Tahmin Süresi: {pred_time:.6f} saniye")
47
48 print(f"\n--- Model Katsayıları (Özelliklerin Etkisi) ---")
49 coefficients = pd.DataFrame(multi_model.coef_, X.columns, columns=['Katsayı (Coefficient)'])
50 print(coefficients.sort_values('Katsayı (Coefficient)', ascending=False))
51
52 # Görselleştirme
53 plt.figure(figsize=(10, 6))
54 sns.scatterplot(x=y_test, y=y_pred_multi, alpha=0.6)
55 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', linewidth=2, label='Mükemmel Tahmin (y=x)')
56 plt.title('Çoklu Linear Regresyon: Gerçek Fiyatlar vs. Tahmin Edilen Fiyatlar')
57 plt.xlabel('Gerçek Ev Fiyatı (y_test)')
58 plt.ylabel('Tahmin Edilen Ev Fiyatı (y_pred)')
59 plt.legend()
60 plt.grid(True)
61 plt.savefig('multiple_linear_regression.png')
62 print("\nGrafik 'multiple_linear_regression.png' olarak kaydedildi.")

```

```

--- Çoklu Linear Regresyon (Tüm Özellikler) ---

--- Performans Metrikleri ---
R-Kare (R2): 0.9984
Ortalama Mutlak Hata (MAE): 8174.58
Kök Ortalama Kare Hata (RMSE): 10071.48

--- Süre Metrikleri ---
Eğitim Süresi: 0.001012 saniye
Tahmin Süresi: 0.000426 saniye

--- Model Katsayıları (Özelliklerin Etkisi) ---
Katsayı (Coefficient)
Num_Bedrooms      10225.204424
Num_Bathrooms      8208.434778
Garage_Size        5146.148383
Year_Built         993.537171
Square_Footage     199.513245
Neighborhood_Quality 115.068595

```

```

polynomial_regression.py
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import numpy as np
4 from sklearn.linear_model import LinearRegression
5 from sklearn.preprocessing import PolynomialFeatures
6 from sklearn.pipeline import make_pipeline
7 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
8 import matplotlib.pyplot as plt
9 import seaborn as sns
10 import time
11
12 # --- VERİ YÜKLEME VE AYIRMA ---
13 file_path = 'house_price_regression_dataset.csv'
14 df = pd.read_csv(file_path)
15 y = df['House_Price']
16 X = df.drop('House_Price', axis=1)
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
18 # --- BİTTİ ---
19
20 print("--- Polinomsal Regresyon (Tüm Özellikler, 2. Derece) ---")
21
22 degree = 2
23 poly_model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
24
25 # --- Eğitim Süresi Ölçümü ---
26 print(f"{degree}. dereceden polinom özellikleri oluşturuluyor ve model eğitiliyor...")
27 start_train = time.time()
28 poly_model.fit(X_train, y_train)
29 train_time = time.time() - start_train
30 # --- BİTTİ ---
31
32 # --- Tahmin Süresi Ölçümü ---
33 print("Tahmin yapılıyor...")
34 start_pred = time.time()
35 y_pred_poly = poly_model.predict(X_test)
36 pred_time = time.time() - start_pred
37 # --- BİTTİ ---
38
39 print(f"{degree}. dereceden polinom özellikleri oluşturuluyor ve model eğitiliyor...")
40 poly_model.fit(X_train, y_train)
41
42 # 2. Test Seti Üzerinde Tahmin Yapma
43 print("Tahmin yapılıyor...")
44 y_pred_poly = poly_model.predict(X_test)
45
46 r2_poly = r2_score(y_test, y_pred_poly)
47 mae_poly = mean_absolute_error(y_test, y_pred_poly)
48 rmse_poly = np.sqrt(mean_squared_error(y_test, y_pred_poly))
49
50 print(f"\n--- Test Seti Performans Metrikleri ---")
51 print(f"R-Kare (R2): {r2_poly:.4f}")
52 print(f"Ortalama Mutlak Hata (MAE): {mae_poly:.2f}")
53 print(f"Kök Ortalama Kare Hata (RMSE): {rmse_poly:.2f}")
54
55 print(f"\n--- Süre Metrikleri ---")
56 print(f"Eğitim Süresi: {train_time:.6f} saniye")
57 print(f"Tahmin Süresi: {pred_time:.6f} saniye")
58
59
60 plt.figure(figsize=(10, 6))
61 sns.scatterplot(x=y_test, y=y_pred_poly, alpha=0.6)
62 # Mükemmel tahmin çizgisi (y=x doğrusu)
63 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', linewidth=2, label='Mükemmel Tahmin (y=x)')
64 plt.title(f"Polinomsal Regresyon (Derece {degree}): Gerçek Fiyatlar vs. Tahminler")
65 plt.xlabel('Gerçek Ev Fiyatı (y_test)')
66 plt.ylabel('Tahmin Edilen Ev Fiyatı (y_pred)')
67 plt.legend()
68 plt.grid(True)
69 plt.savefig('polynomial_regression.png')
70 print("\nGrafik 'polynomial_regression.png' olarak kaydedildi.")

```

```

PS C:\Users\Lab2-19\Desktop\0dev> python .\polynomial_regression.py
--- Polinomsal Regresyon (Tüm Özellikler, 2. Derece) ---
2. dereceden polinom özellikleri oluşturuluyor ve model eğitiliyor...
Tahmin yapılıyor...
2. dereceden polinom özellikleri oluşturuluyor ve model eğitiliyor...
Tahmin yapılıyor...

--- Test Seti Performans Metrikleri ---
R-Kare (R2): 0.9984
Ortalama Mutlak Hata (MAE): 8311.02
Kök Ortalama Kare Hata (RMSE): 10187.32

--- Süre Metrikleri ---
Eğitim Süresi: 0.003000 saniye

--- Test Seti Performans Metrikleri ---
R-Kare (R2): 0.9984
Ortalama Mutlak Hata (MAE): 8311.02
Kök Ortalama Kare Hata (RMSE): 10187.32

```

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 import numpy as np
4 # Hata düzeltmesi: LinearRegression kaldırıldı, DecisionTreeRegressor eklendi
5 from sklearn.tree import DecisionTreeRegressor
6 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9 import time
10
11 # --- VERİ YÜKLEME VE AYIRMA ---
12 file_path = 'house_price_regression_dataset.csv'
13 df = pd.read_csv(file_path)
14 y = df['House_Price']
15 X = df.drop('House_Price', axis=1)
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
17 # --- BİTTİ ---
18
19 print("--- Karar Ağacı Regresyonu (Decision Tree) ---")
20
21 # --- HATA DÜZELTMESİ: Yanlış modele ait süre ölçümü kaldırıldı ---
22 # LinearRegression() ile ilgili kod blokları silindi.
23
24 # 1. Modeli Oluşturma ve Eğitim (Süre ölçümü ile birlikte)
25 print("Karar Ağacı modeli eğitiliyor...")
26 start_train = time.perf_counter()
27 tree_model = DecisionTreeRegressor(random_state=42)
28 tree_model.fit(X_train, y_train)
29 train_time = time.perf_counter() - start_train
30 # --- BİTTİ ---
31
32 # 2. Test Seti Üzerinde Tahmin Yapma (Süre ölçümü ile birlikte)
33 print("Tahmin yapılıyor...")
34 start_pred = time.perf_counter()
35 y_pred_tree = tree_model.predict(X_test)
36 pred_time = time.perf_counter() - start_pred
37 # --- BİTTİ ---
38
39 # 3. Model Performansını Değerlendirme
40 r2_tree = r2_score(y_test, y_pred_tree)
41 mae_tree = mean_absolute_error(y_test, y_pred_tree)
42 rmse_tree = np.sqrt(mean_squared_error(y_test, y_pred_tree))
43
44 print(f"\n--- Test Seti Performans Metrikleri ---")
45 print(f"R-Kare (R2): {r2_tree:.4f}")
46 print(f"Ortalama Mutlak Hata (MAE): {mae_tree:.2f}")
47 print(f"Kök Ortalama Kare Hata (RMSE): {rmse_tree:.2f}")
48
49 print(f"\n--- Süre Metrikleri ---")
50 # Süreleri daha hassas göstermek için .8f kullanıldı
51 print(f"Eğitim Süresi: {train_time:.8f} saniye")
52 print(f"Tahmin Süresi: {pred_time:.8f} saniye")
53
54 # 4. Görselleştirme: Gerçek Değerler vs. Tahmin Edilen Değerler
55 plt.figure(figsize=(10, 6))
56 sns.scatterplot(x=y_test, y=y_pred_tree, alpha=0.6)
57 # Mükemmel tahmin çizgisi (y=x doğrusu)
58 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', linewidth=2, label='Mükemmel Tahmin (y=x)')
59 plt.title('Karar Ağacı Regresyonu: Gerçek Fiyatlar vs. Tahmin Edilen Fiyatlar')
60 plt.xlabel('Gerçek Ev Fiyatı (y_test)')
61 plt.ylabel('Tahmin Edilen Ev Fiyatı (y_pred)')
62 plt.legend()
63 plt.grid(True)
64 plt.savefig('decision_tree_regression.png')
65 print("\nGrafik 'decision_tree_regression.png' olarak kaydedildi.")

```

```

--- Karar Ağacı Regresyonu (Decision Tree) ---
Karar Ağacı modeli eğitiliyor...
--- Karar Ağacı Regresyonu (Decision Tree) ---
Karar Ağacı modeli eğitiliyor...
Tahmin yapılıyor...
Karar Ağacı modeli eğitiliyor...
Tahmin yapılıyor...

--- Test Seti Performans Metrikleri ---
Tahmin yapılıyor...

--- Test Seti Performans Metrikleri ---

--- Test Seti Performans Metrikleri ---
R-Kare (R2): 0.9850
Ortalama Mutlak Hata (MAE): 24084.46
Ortalama Mutlak Hata (MAE): 24084.46
Kök Ortalama Kare Hata (RMSE): 31138.40
Kök Ortalama Kare Hata (RMSE): 31138.40

--- Süre Metrikleri ---
Eğitim Süresi: 0.00341620 saniye
Tahmin Süresi: 0.00051790 saniye

```